

# DealFlow

## Enterprise CPQ Deal Desk & Approval Automation Platform

### Project Specification, Architecture, Development Plan, and Interview Preparation

|                          |                                              |
|--------------------------|----------------------------------------------|
| <b>Consultant:</b>       | Vamshi Kasarla                               |
| <b>Role:</b>             | Oracle CPQ Cloud – Freelance Consultant      |
| <b>Project Duration:</b> | Jul 2025 – Oct 2025                          |
| <b>Client Scenario:</b>  | Apex Industrial Solutions                    |
| <b>Client Contact:</b>   | Daniel Carter – Director of Sales Operations |

---

## Important Project Note

This document is a technical project specification designed to guide the implementation of a realistic enterprise Oracle CPQ integration project.

The client name “Apex Industrial Solutions” and contact “Daniel Carter” are fictional placeholders unless they correspond to a real engagement. They should not be represented as real clients if they were not real.

The objective is to build the system described here and gain genuine hands-on understanding of every technology, architectural decision, and business workflow used in the project.

## 1. Executive Summary

DealFlow is an enterprise Deal Desk and approval automation platform designed around Oracle CPQ Cloud.

The platform addresses a common enterprise quoting problem: Oracle CPQ can manage product configuration, pricing, and quote transactions, but organizations may require additional workflow capabilities around discount approvals, margin exceptions, management review, and audit visibility.

The proposed platform keeps Oracle CPQ as the primary system for product configuration and quote transaction information while introducing a custom application responsible for deal review and approval orchestration.

The system consists of:

- Oracle CPQ Cloud
- React frontend
- Spring Boot backend
- PostgreSQL database
- REST API integration
- JWT authentication
- Spring Security
- Role-Based Access Control (RBAC)
- Configurable approval rules engine

- Exception management
- Audit logging
- Docker containerization
- GitHub Actions CI
- Automated testing

## 2. Client Background

Apex Industrial Solutions is modeled as an enterprise company that sells configurable networking and infrastructure equipment.

The sales organization uses Oracle CPQ Cloud to:

- Configure products
- Select product attributes
- Calculate pricing
- Apply discounts
- Generate quotes
- Manage Commerce transactions

The sales organization has several approval levels.

Typical users include:

- Sales Representatives
- Sales Managers
- Finance Analysts
- VP Sales
- CPQ Administrators

## 3. Client Problem

The client identified that the quote creation process was not the only challenge.

The primary issue was the process used after a quote required additional approval.

For example, a Sales Representative could create a quote with:

Customer: Acme Corporation  
Quote: Q-10482

Deal Value: \$520,000  
Discount: 25%  
Expected Margin: 13%

The quote could require approval from:

Sales Manager

|

v

Finance

|

v

VP Sales

The existing process required people to manually coordinate these approvals.

The client wanted a centralized system that could determine:

- Whether approval is required
- Which roles must approve
- In what order approvals should occur
- Why the approval was triggered
- Whether an exception exists
- Who approved or rejected the deal
- When the decision occurred

## 4. Business Objectives

The main objectives are:

1. Reduce manual approval coordination.
2. Provide centralized visibility into deal status.
3. Integrate Oracle CPQ quote information with the Deal Desk.
4. Automatically determine approval requirements.
5. Support pricing and margin exceptions.
6. Provide role-specific approval queues.
7. Maintain a complete audit trail.
8. Secure sensitive deal information.
9. Provide automated testing and CI.

## 5. Project Scope

### 5.1 In Scope

- Oracle CPQ product configuration
- CPQ product models
- CPQ attributes

- CPQ pricing configuration
- CPQ Commerce transactions
- Oracle CPQ REST API integration
- Spring Boot REST APIs
- PostgreSQL database
- React dashboard
- Approval workflow
- Approval rules engine
- Exception management
- JWT authentication
- RBAC
- Audit logging
- Docker
- GitHub Actions
- Unit testing
- Integration testing
- API testing

## 5.2 Out of Scope

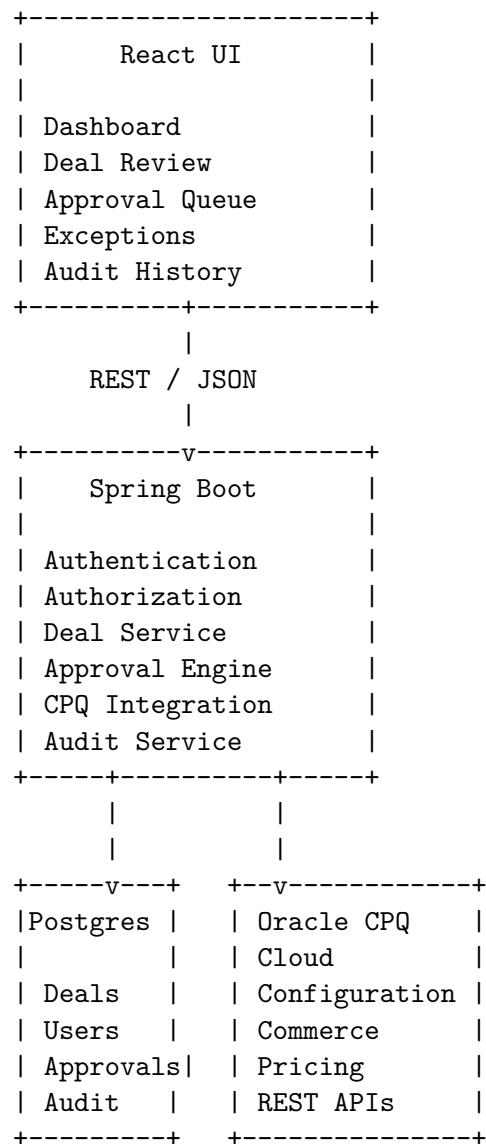
The initial version does not attempt to replace Oracle CPQ.

The following are intentionally outside the first release:

- Rebuilding the entire Oracle CPQ platform
- Replacing Oracle pricing calculations
- Replacing the Oracle CPQ product configurator
- Full enterprise identity management
- Production-scale Kubernetes infrastructure
- Complex event streaming
- Full ERP replacement

## 6. High-Level Architecture

The system follows a layered architecture.



## 7. Component Responsibilities

### 7.1 Oracle CPQ Cloud

Oracle CPQ is responsible for:

- Product configuration
- Configurable attributes
- Product structures
- Bill of Materials
- Pricing
- Quote transaction information
- CPQ Commerce process

## 7.2 Spring Boot

Spring Boot is responsible for:

- REST APIs
- Business logic
- Approval rules
- Workflow state transitions
- CPQ integration
- Security
- Database access
- Audit logging

## 7.3 React

React is responsible for:

- User interface
- Dashboards
- Deal review
- Approval queues
- Exception forms
- Audit history

## 7.4 PostgreSQL

PostgreSQL stores application-owned data such as:

- Users
- Roles
- Deals
- Approval requests
- Approval decisions
- Exceptions
- Audit events

## 8. Oracle CPQ Product Catalog

The demonstration catalog is:

```
Apex Networking
|
+--- Enterprise Router
|
+--- Security Gateway
|
+--- Branch Router
```

## 8.1 Enterprise Router Attributes

- Bandwidth
- Support Level
- Warranty
- Deployment Type

Example configuration:

Product:  
Enterprise Router

Bandwidth:  
10 Gbps

Support:  
Premium

Warranty:  
5 Years

Deployment:  
On-Premise

## 9. Oracle CPQ Configuration Rules

Configuration rules prevent invalid combinations.

Example:

```
IF bandwidth = "10 Gbps"
AND deployment = "Branch"
THEN configuration is invalid
```

Another example:

```
IF supportLevel = "Premium"
THEN warranty options include 5 Years
```

The objective is to understand the distinction between:

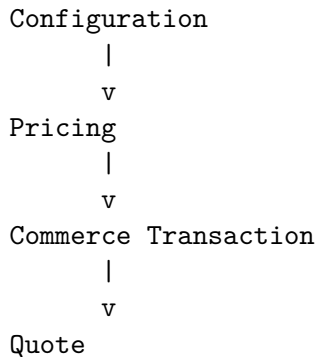
- Configuration rules
- Pricing rules

- Commerce rules
- Approval rules

## 10. Oracle CPQ Commerce Transaction

After configuration and pricing, the quote exists as a Commerce transaction.

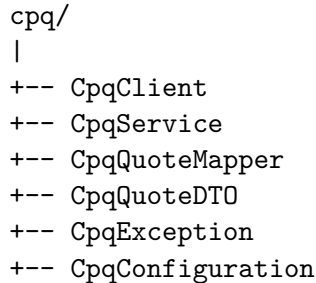
Conceptually:



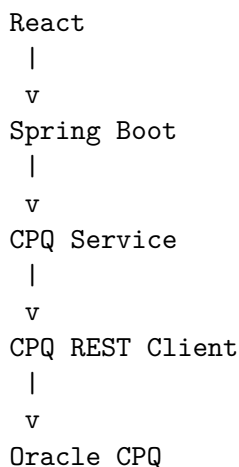
The external DealFlow application retrieves relevant transaction information through the CPQ REST API.

## 11. Oracle CPQ REST Integration

The integration is implemented as a dedicated Spring Boot module.



The integration follows:



The frontend never directly calls Oracle CPQ.

This provides:

- Credential protection
- Centralized error handling
- Data transformation
- Logging
- API abstraction

## 12. DTO Mapping

Oracle CPQ response structures are not exposed directly to the frontend.

Example internal DTO:

```
public class CpqQuoteDTO {
    private String quoteNumber;
    private String customerName;
    private BigDecimal totalPrice;
    private BigDecimal discount;
    private BigDecimal margin;
    private String status;
}
```

The flow is:

```
Oracle CPQ JSON
  |
  v
CPQ Response Model
  |
  v
CpqqQuoteDTO
  |
  v
DealFlow Domain Model
```

## 13. Deal Lifecycle

A deal follows these states:

```
DRAFT
  |
  v
SUBMITTED
  |
  v
MANAGER_REVIEW
  |
  v
FINANCE_REVIEW
  |
```

```

    v
VP_REVIEW
    |
    v
APPROVED

```

Alternative states include:

```

REJECTED
CHANGES_REQUESTED
CANCELLED

```

## 14. Approval Rules Engine

The approval engine is one of the main components of the application.

The engine receives deal information:

```

Deal Value
Discount
Margin
Product Category
Exception Type

```

It evaluates configured business rules.

Example:

```

Discount < 10%
    |
    +--- No additional approval

10% <= Discount <= 20%
    |
    +--- Sales Manager

20% < Discount <= 30%
    |
    +--- Sales Manager
    +--- Finance

Discount > 30%
    |
    +--- Sales Manager
    +--- Finance
    +--- VP Sales

```

Margin rule:

```

IF margin < 15%
THEN Finance approval

```

Deal-value rule:

```

IF dealValue > $500,000
THEN VP Sales approval

```

## 15. Approval Engine Design

The approval engine should not hardcode every possible workflow.

Instead, rules should be represented as configurable application data.

Conceptually:

```
Deal
|
v
Rule Evaluation
|
+-- Discount Rule
|
+-- Margin Rule
|
+-- Deal Value Rule
|
+-- Exception Rule
|
v
Approval Requirements
|
v
Approval Chain
```

Example result:

```
{
  "requiredApprovals": [
    "SALES_MANAGER",
    "FINANCE",
    "VP_SALES"
  ]
}
```

## 16. Exception Management

Not every policy violation should automatically reject a quote.

Instead, DealFlow supports exceptions.

Example:

Standard Discount Limit = 20%

Requested Discount = 27%

```

|
v

Policy Exception
|
v
```

Sales Justification

|

v

Manager Review

|

v

Finance Review

A Sales Representative can provide a justification:

Customer committed to a three-year contract and competitor pricing is approximately 18 percent lower.

The approver can:

- Approve
- Reject
- Request Changes

## 17. Users and Roles

The initial role model is:

| Role          | Responsibilities                                 |
|---------------|--------------------------------------------------|
| SALES_REP     | Create and submit deals, view own deal status    |
| SALES_MANAGER | Review and approve sales deals                   |
| FINANCE       | Review financial exceptions and margin issues    |
| VP_SALES      | Approve high-value and high-discount deals       |
| CPQ_ADMIN     | Manage integration and application configuration |

## 18. Security Architecture

Authentication uses JWT.

The basic authentication flow is:

User

|

v

Login

|

v

Spring Security

|

v

Credentials Validated

|

v

JWT Issued

|

v

```

React
|
v
Authorization Header
|
v
Spring Boot

```

Example:

Authorization: Bearer <JWT>

The backend validates the token before processing protected requests.

## 19. Role-Based Access Control

RBAC is enforced on the backend.

Example:

POST /api/approvals/{id}/approve

A Sales Representative should receive:

```

\begin{verbatim}
403 FORBIDDEN

```

if they attempt to approve an approval request that requires Manager, Finance, or VP authorization.

The frontend may hide buttons for convenience, but the backend remains the authoritative authorization layer.

## 20. Database Design

The application database uses PostgreSQL.

Core entities include:

```

users
roles
user\_roles
deals
deal\_items
approval\_requests
approval\_steps
exceptions
audit\_events

```

### 20.1 Users Table

```

id
first\_name
last\_name
email
password\_hash

```

```
status
created\_at
updated\_at
```

## 20.2 Deals Table

```
id
quote\_number
customer\_name
deal\_value
discount\_percentage
margin\_percentage
status
created\_by
created\_at
updated\_at
```

## 20.3 Approval Requests

```
id
deal\_id
approver\_role
assigned\_user\_id
status
sequence
requested\_at
resolved\_at
comments
```

## 20.4 Exceptions

```
id
deal\_id
exception\_type
requested\_value
standard\_value
justification
status
created\_by
created\_at
resolved\_at
```

## 20.5 Audit Events

```
id
deal\_id
user\_id
event\_type
old\_status
new\_status
```

```

comments
metadata
created\_at

```

## 21. Database Relationships

Conceptually:

```

User
|
+-----< Deal
|
+-----< Audit Event
|
+-----< Approval Request

```

```

Deal
|
+-----< Deal Item
|
+-----< Approval Request
|
+-----< Exception
|
+-----< Audit Event

```

## 22. REST API Design

The backend exposes REST endpoints.

### 22.1 Authentication

```

POST /api/auth/login
POST /api/auth/refresh

```

### 22.2 Deals

```

GET /api/deals
GET /api/deals/{id}
POST /api/deals
POST /api/deals/{id}/submit
PUT /api/deals/{id}

```

### 22.3 Approvals

```

GET /api/approvals
GET /api/approvals/{id}
POST /api/approvals/{id}/approve
POST /api/approvals/{id}/reject
POST /api/approvals/{id}/request-changes

```

## 22.4 Exceptions

```
GET /api/deals/{id}/exceptions
POST /api/deals/{id}/exceptions
POST /api/exceptions/{id}/approve
POST /api/exceptions/{id}/reject
```

## 22.5 Audit

```
GET /api/deals/{id}/audit
```

## 23. API Example

Submitting a deal:

```
POST /api/deals/10482/submit
```

The backend retrieves deal information:

```
{
  "dealValue": 520000,
  "discount": 25,
  "margin": 13
}
```

The approval engine evaluates the rules.

Result:

```
{
  "status": "SUBMITTED",
  "approvalChain": [
    "SALES\_MANAGER",
    "FINANCE",
    "VP\_SALES"
  ]
}
```

## 24. React Frontend

The React application contains:

```
/src
|
+-- components
+-- pages
+-- services
+-- hooks
+-- auth
+-- routing
+-- layouts
+-- utils
```

Main pages:

```
/login
/dashboard
/deals
/deals/\:id
/approvals
/exceptions
/audit
/admin/rules
```

## 25. Deal Desk Dashboard

The dashboard displays:

DEAL DESK

```
Pending Approvals 12
Finance Reviews 5
High Risk Deals 3
Approved This Month 27
```

---

Customer Value Discount Status

Acme Corp \$520K 25% Finance

GlobalTech \$310K 14% Manager

TechWorld \$750K 32% VP

## 26. Deal Details Page

The deal details page displays:

- Customer
- Quote number
- Products
- Configuration
- Total price
- Discount
- Margin
- Exception information
- Approval chain

- Audit history

Example:

Deal: Acme Network Upgrade

Customer:

Acme Corporation

Quote:

Q-10482

Value:

\$520,000

Discount:

25%

Margin:

13%

Approval Chain:

☒ Sales Manager

☐ Finance

☐ VP Sales

## 27. Audit History

The audit page displays events chronologically.

Example:

10:21 AM

John created quote

10:27 AM

Quote synchronized from Oracle CPQ

10:30 AM

Discount approval rule triggered

10:31 AM

Manager approval requested

11:04 AM

Manager approved

11:07 AM

Finance approval requested

12:15 PM

Finance approved

12:20 PM

VP approval requested

12:45 PM

VP approved

12:46 PM

Deal approved

## 28. Client Communication Story

The project should follow a realistic consulting workflow.

### 28.1 Initial Client Email

**Subject: CPQ Deal Approval Workflow**

Hi Daniel,

Based on our initial discussion, I understand that the main challenge is the manual coordination required for quotes that exceed standard discount or margin thresholds.

I propose building a Deal Desk application around Oracle CPQ that will centralize approval routing, exception handling, and audit visibility.

The first phase will focus on understanding the existing CPQ Commerce workflow and defining the approval requirements.

Thanks,

Vamshi

### 28.2 Requirements Update

**Subject: DealFlow – Initial Requirements**

Hi Daniel,

I've documented the initial requirements for the DealFlow application.

The first version will include:

- Oracle CPQ quote synchronization
- Deal dashboard
- Discount approval routing
- Margin-based Finance review
- Exception management
- Role-based access
- Approval audit history

I'll proceed with the initial CPQ investigation and API integration design.

Thanks,

Vamshi

## 28.3 CPQ Integration Update

### **Subject: Oracle CPQ API Investigation**

Hi Daniel,

I've completed the initial investigation of the CPQ API integration.

The integration layer will normalize CPQ transaction data before exposing it to the Deal Desk application. This keeps CPQ-specific response structures isolated from the application's internal business logic.

Thanks,

Vamshi

## 29. Client Requirement Change

During testing, the client identifies an additional business rule.

Original logic:

`Discount = 25%`

`Margin = 12%`

Required:

`Manager`

`Finance`

The client explains that deals exceeding \$500,000 also require VP approval.

The requirement becomes:

`IF dealValue > $500,000`

`THEN VP\_SALES approval`

The rule is implemented and an automated test is added.

This demonstrates how the system handles evolving business requirements.

## 30. Sixteen-Week Development Timeline

### 30.1 Weeks 1–2: Discovery and Requirements

Activities:

- Client discussions
- Business process analysis
- Approval hierarchy definition
- Discount threshold definition

- Margin policy analysis
- Exception requirements
- Initial architecture

Deliverables:

- Requirements document
- Architecture diagram
- Initial data model

### **30.2 Weeks 3–4: Oracle CPQ Configuration**

Activities:

- Product model configuration
- Attribute configuration
- Product structures
- BOM configuration
- Configuration rules
- Pricing rules
- Commerce workflow testing

Deliverables:

- Working CPQ catalog
- Test configurations
- Sample quote transactions

### **30.3 Week 5: REST API Investigation**

Activities:

- CPQ REST API investigation
- Postman testing
- Authentication testing
- JSON response analysis
- DTO design
- Error handling design

Deliverables:

- API test collection

- CPQ integration specification

### 30.4 Weeks 6–7: Spring Boot Backend

Activities:

- Spring Boot project setup
- REST API development
- PostgreSQL integration
- Entity modeling
- DTOs
- Service layer
- Repository layer
- CPQ client

Deliverables:

- Backend API
- Database schema
- CPQ integration service

### 30.5 Week 8: Security

Activities:

- JWT authentication
- Spring Security
- Role model
- Authorization
- Protected endpoints
- Permission testing

Deliverables:

- Secure API
- RBAC implementation

### 30.6 Weeks 9–10: Approval Engine

Activities:

- Approval rule implementation
- Discount rules

- Margin rules
- Deal-value rules
- Approval sequencing
- State transitions
- Automated tests

Deliverables:

- Approval engine
- Approval workflow
- Rule test suite

### **30.7 Week 11: Client Testing**

Activities:

- End-to-end testing
- Requirement clarification
- New deal-value rule
- Regression testing

Deliverables:

- Updated rules
- Regression tests

### **30.8 Weeks 12–13: React Application**

Activities:

- React application setup
- Authentication screens
- Dashboard
- Deal list
- Deal details
- Approval queue
- Exception page
- Audit page

Deliverables:

- Working Deal Desk UI

- Role-specific views

### **30.9 Week 14: Audit Trail**

Activities:

- Audit event model
- Status transition logging
- Approval event logging
- User activity tracking
- Audit UI

Deliverables:

- Complete deal audit trail

### **30.10 Week 15: Docker and CI**

Activities:

- Dockerfile creation
- Docker Compose
- PostgreSQL container
- Backend container
- Frontend container
- GitHub Actions
- Automated testing

Deliverables:

- Containerized application
- CI workflow

### **30.11 Week 16: Final Demonstration**

Activities:

- End-to-end testing
- Client demonstration
- Approval workflow validation
- Security validation
- Audit validation
- Documentation

Deliverables:

- Final application
- Technical documentation
- Demo environment

## 31. Testing Strategy

Testing is divided into multiple levels.

### 31.1 Unit Testing

Unit tests focus on business logic.

Example:

`discount = 5%`

`Expected = no approval`

`discount = 15%`

`Expected = Sales Manager`

`discount = 25%`

`Expected = Sales Manager + Finance`

`discount = 35%`

`Expected = Sales Manager + Finance + VP Sales`

### 31.2 Integration Testing

Integration tests verify communication between:

`Spring Boot`

`|`

`--- PostgreSQL`

`|`

`--- CPQ integration layer`

### 31.3 Security Testing

Examples:

`Sales Rep -> Approve Finance Request`

`Expected: 403`

`Unauthenticated User -> GET /api/deals`

`Expected: 401`

### 31.4 Workflow Testing

Example:

```

Create Deal
|
v
Submit
|
v
Manager Approves
|
v
Finance Approves
|
v
VP Approves
|
v
APPROVED

```

## 32. Docker Architecture

The application is containerized into services.

```

+-----+
\| React Container |
+-----+-----+
|
v
+-----+
\| Spring Boot Container |
+-----+-----+
|
v
+-----+
\| PostgreSQL Container |
+-----+-----+

```

Docker Compose manages the local environment.

Example structure:

`docker-compose.yml`

```

services:
frontend
backend
postgres

```

## 33. GitHub Actions CI

The CI pipeline executes after a Git push.

```

Git Push
|
v

```

```
Checkout
|
v
Build Backend
|
v
Run Unit Tests
|
v
Build Frontend
|
v
Integration Tests
|
v
Docker Build
```

The goal is to prevent broken code from reaching the next environment.

## 34. Environment Configuration

Sensitive and environment-specific configuration should not be hardcoded.

Example variables:

```
DATABASE\_URL
DATABASE\_USERNAME
DATABASE\_PASSWORD
```

```
CPQ\_BASE\_URL
CPQ\_USERNAME
CPQ\_PASSWORD
```

```
JWT\_SECRET
```

Development and production configurations should remain separate.

## 35. Application Logging

The backend should provide structured logging for important events.

Examples:

```
CPQ API request started
CPQ API response received
Deal submitted
Approval rule evaluated
Approval request created
Approval completed
Authentication failed
```

Sensitive credentials and tokens must never be written to logs.

## 36. Error Handling

The backend should use centralized exception handling.

Examples:

400 BAD REQUEST  
Invalid deal data

401 UNAUTHORIZED  
Missing or invalid authentication

403 FORBIDDEN  
Insufficient permissions

404 NOT FOUND  
Deal does not exist

409 CONFLICT  
Invalid workflow transition

500 INTERNAL SERVER ERROR  
Unexpected application failure

## 37. Workflow Validation

The backend prevents invalid transitions.

Valid:

DRAFT -> SUBMITTED  
SUBMITTED -> MANAGER\\_REVIEW  
MANAGER\\_REVIEW -> FINANCE\\_REVIEW  
FINANCE\\_REVIEW -> VP\\_REVIEW  
VP\\_REVIEW -> APPROVED

Invalid:

DRAFT -> APPROVED

The backend rejects invalid transitions regardless of frontend behavior.

```
% =====
% 45. PERFORMANCE CONSIDERATIONS
% =====
```

```
\section{Performance Considerations}
```

The first implementation focuses on correctness rather than premature optimization.

Potential improvements include:

```
\begin{itemize}
  \item Database indexing
  \item Pagination
  \item API response optimization
  \item Connection pooling
  \item Caching
  \item Asynchronous processing where appropriate
\end{itemize}
```

For example, frequently accessed dashboard data can later be cached.

```
% =====
% 46. SECURITY CONSIDERATIONS
% =====
```

```
\section{Security Considerations}
```

Security requirements include:

```
\begin{itemize}
  \item Password hashing
  \item JWT validation
  \item Role-based authorization
  \item HTTPS in deployed environments
  \item Secure secret management
  \item Input validation
  \item SQL injection protection through parameterized queries / ORM
  \item CORS configuration
  \item Audit logging
  \item No sensitive credentials in source control
\end{itemize}
```

```
% =====
% 47. GIT STRATEGY
% =====
```

```
\section{Git Workflow}
```

A simple Git workflow is used.

```
\begin{verbatim}
main
|
+-- develop
    |
    +-- feature/cpq-integration
    +-- feature/approval-engine
    +-- feature/security
    +-- feature/react-dashboard
    +-- feature/audit
```

Each feature is developed independently and merged after testing.

## 38. Repository Structure

A possible repository structure is:

```
dealflow/
|
+-- frontend/
|   |
|   +-- src/
|   +-- package.json
|   +-- Dockerfile
|
+-- backend/
|   |
|   +-- src/
|   +-- pom.xml
|   +-- Dockerfile
|
+-- database/
|   |
|   +-- migrations/
|
+-- docker-compose.yml
|
+-- .github/
|   |
|   +-- workflows/
|
+-- docs/
|   |
|   +-- architecture/
|   +-- api/
|   +-- requirements/
|
+-- README.md
```

## 39. Technical Skills Demonstrated

### 39.1 Oracle CPQ

- Oracle CPQ Cloud
- Product configuration
- Configurable attributes
- Product models
- BOM
- Configuration rules

- Pricing
- Commerce transactions
- CPQ REST APIs

### **39.2 Backend**

- Java
- Spring Boot
- Spring Security
- REST APIs
- JSON
- DTOs
- Business logic
- Exception handling

### **39.3 Frontend**

- React
- JavaScript / TypeScript
- REST API integration
- Authentication state
- Role-based UI

### **39.4 Database**

- PostgreSQL
- SQL
- Relational modeling
- Transactions
- Indexing

### **39.5 Security**

- JWT
- Spring Security
- RBAC
- Authentication
- Authorization

## 39.6 DevOps

- Docker
- Docker Compose
- GitHub Actions
- CI
- Environment configuration

## 39.7 Testing

- JUnit
- Mockito
- Integration testing
- REST API testing
- Postman

# 40. Major Project Challenges

## 40.1 Challenge 1 – Separating CPQ and DealFlow Responsibilities

The first architectural challenge is deciding what belongs in Oracle CPQ and what belongs in the custom application.

Solution:

Oracle CPQ remains responsible for product configuration, pricing, and Commerce transactions, while DealFlow handles approval orchestration, exceptions, audit history, and user-facing deal review.

## 40.2 Challenge 2 – Dynamic Approval Routing

Approval requirements vary based on multiple factors.

For example:

Discount

Margin

Deal Value

Exception Type

The solution is a configurable rules engine rather than hardcoded workflow branches.

## 40.3 Challenge 3 – Authorization

The system must prevent users from performing actions outside their roles.

The solution is backend-enforced RBAC using Spring Security.

#### 40.4 Challenge 4 – CPQ API Response Mapping

Oracle CPQ responses contain fields that are not required by the application.

The solution is to use DTOs and a dedicated mapping layer.

#### 40.5 Challenge 5 – Changing Client Requirements

The client introduces a deal-value approval requirement during testing.

The system is designed so that a new rule can be introduced without rewriting the entire approval workflow.

#### 40.6 Challenge 6 – Auditability

Approval actions must remain traceable.

The solution is an immutable audit-event model that records:

- User
- Action
- Timestamp
- Previous status
- New status
- Comments

### 41. Final Demonstration Scenario

The final demonstration uses the following quote:

Customer:

Acme Corporation

Quote:

Q-10482

Deal Value:

\$520,000

Discount:

25%

Margin:

13%

The system evaluates:

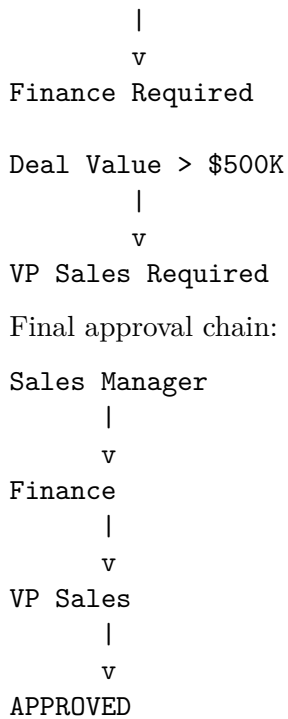
Discount > 20%

|

v

Finance Required

Margin < 15%



The audit trail records every action.

## 42. Expected Business Outcome

The completed application provides:

- Centralized deal visibility
- Automated approval routing
- Structured exception handling
- Role-specific approval queues
- CPQ integration
- Auditability
- Secure access
- Automated testing

The core business improvement is reducing the need for sales teams to manually coordinate approval status through email and disconnected processes.

## 43. Interview Preparation

The following questions should be answerable after completing the project.

### 43.1 Architecture

1. Why did you use Spring Boot?

2. Why did you separate DealFlow from Oracle CPQ?
3. Why didn't you rebuild the CPQ functionality?
4. Why did you use PostgreSQL?
5. Why did you use React?

### **43.2 Oracle CPQ**

1. What is Oracle CPQ?
2. What is a configurable product model?
3. What are configurable attributes?
4. What is a BOM?
5. What are configuration rules?
6. What are Commerce transactions?
7. How did you integrate with CPQ?
8. Why did you use REST APIs?

### **43.3 Backend**

1. How is your Spring Boot application structured?
2. What is the purpose of DTOs?
3. How do you handle exceptions?
4. How do you validate requests?
5. How does the approval engine work?

### **43.4 Security**

1. How does JWT authentication work?
2. What is the difference between authentication and authorization?
3. How did you implement RBAC?
4. Why can't a Sales Representative approve their own deal?
5. Why must authorization be enforced on the backend?

### **43.5 Database**

1. Why PostgreSQL?
2. What are the relationships between deals and approvals?
3. How did you model audit events?
4. How would you index the deals table?

## 43.6 DevOps

1. Why Docker?
2. What does Docker Compose provide?
3. What does your GitHub Actions workflow do?
4. How do you manage environment variables?

## 44. Two-Minute Project Explanation

A concise explanation of the project is:

I worked on an enterprise Deal Desk and approval automation platform designed around Oracle CPQ Cloud.

The client was using Oracle CPQ for configurable product quoting, but the approval process for high-discount, low-margin, and high-value deals was largely manual.

I designed the solution so Oracle CPQ remained the system responsible for product configuration, pricing, and Commerce transactions. I built a Spring Boot application around it to handle deal review, approval orchestration, exception management, and audit history.

I integrated Oracle CPQ REST APIs with Spring Boot and created DTOs to normalize the CPQ transaction data.

One of the main components I developed was a configurable approval rules engine. It evaluated discount percentage, margin, and deal value and dynamically generated the required approval chain for Sales Managers, Finance, and VP Sales.

I also built a React dashboard for deal review and implemented JWT authentication and role-based access control using Spring Security.

Finally, I containerized the application using Docker and configured GitHub Actions for automated builds and testing.

## 45. Development Principle

The most important objective of this project is not simply to produce resume keywords.

The objective is to understand the complete system.

Every technology listed on the resume should correspond to something implemented in the project.

For example:

```
Resume Skill
  |
  v
Actual Implementation
  |
  v
Testing
  |
  v
Interview Explanation
```

The project should therefore be developed incrementally.

Do not add technologies such as Kafka, Kubernetes, Redis, AWS, or other infrastructure components unless they solve a real requirement and are actually implemented and understood.

## 46. Final Project Checklist

### 46.1 Oracle CPQ

- Product model configured
- Attributes configured
- Product structure configured
- BOM configured
- Configuration rules tested
- Pricing rules tested
- Commerce workflow tested
- REST APIs tested

### 46.2 Backend

- Spring Boot application
- REST controllers
- Service layer
- Repository layer
- DTOs
- CPQ integration
- Approval engine
- Exception handling

### 46.3 Security

- JWT authentication
- Password hashing
- Spring Security
- RBAC
- Protected endpoints
- Authorization tests

## 46.4 Frontend

- Login
- Dashboard
- Deal list
- Deal details
- Approval queue
- Exception management
- Audit history

## 46.5 Database

- PostgreSQL
- Users
- Roles
- Deals
- Approvals
- Exceptions
- Audit events

## 46.6 DevOps

- Docker
- Docker Compose
- Environment configuration
- GitHub Actions
- Automated testing

---

**DealFlow – Enterprise CPQ Deal Desk & Approval Automation Platform**

*Project Specification for Development and Interview Preparation*

**Vamshi Kasarla**